

Godot C# 类《空洞骑士》2D 像素游 戏架构路线与进度规划

基于当前仓库结构: Resources / Scenes / Scripts

参考模块: Combat / Core / Entities / Level / Player / UI

架构分层 技术路线 Sprint 计划 风险控制

文档版本: V0.1
适用阶段: 原型期/ MVP / 垂直切片
团队假设: 3 名C# 程序员兼职开发
整理日期: 2026年4月24日

1 执行摘要

最优路线：不要直接开发完整商业体量。先基于当前目录结构，把**Combat + Player + Level**三个模块打通，做出“战斗手感+ 4 到6 个房间+ 1 个小Boss + 存档/地图/UI”的垂直切片Demo。该Demo 是继续投入的立项闸门：若3 人兼职在24 到30 周内无法做出外部玩家可独立跑完的20 到30 分钟试玩版本，则应先压缩范围或转为技术Demo。

1.1 关键结论

- 当前仓库第一层分区是合理的：**Core**、**Combat**、**Entities**、**Level**、**Player**、**UI** 能支撑MVP。
- 下一步重点不是继续加功能，而是建立“组件+ 数据资源+ 状态机+ 事件总线”的可维护框架。
- 类《空洞骑士》的最大风险是内容规模和手感，不是Godot 或C# 本身。
- AI 应作为生产辅助：代码骨架、策划表、关卡草图、占位美术、文档与测试用例；最终战斗调参、像素动画统一、Boss 节奏仍需人工负责。

2 当前结构解读

根据截图，仓库结构可抽象为：

```
HollowKnight/  
  Resources/  
  Scenes/  
    mainScene.tscn  
  Scripts/  
    Combat/  
      Hurtbox.cs  
      IDamageable.cs  
      Readme.txt  
    Core/  
    Entities/  
    Level/  
    Player/  
    UI/  
  .editorconfig  
  .gitattributes  
  .gitignore  
  icon.svg
```

2.1 当前优点

- 已经把战斗、核心、实体、关卡、玩家、UI 分离，方向正确。
- **Hurtbox.cs** 与 **IDamageable.cs** 说明团队已经开始抽象“可受伤对象”。这是动作游戏的正确起点。
- C# 程序员可以较自然地使用接口、组件、状态机、数据结构和测试思维。

2.2 立即需要补齐的缺口

- **主场景入口**：将`mainScene.tscn`拆分为`Boot.tscn`、`GameRoot.tscn`、测试场景和房间场景，避免后续所有逻辑堆进单一场景。
- **Combat 数据模型**：将`TakeDamage(int, Vector2)`升级为`DamageInfo`、`DamageTeam`、`AttackProfile`，避免参数继续膨胀。
- **信号定义**：当前`EmitSignal(...)` 需要对应C# `Signal` 声明；复杂数据建议走方法或C# `event`。
- **实体抽象**：建立`EntityBase`、`EnemyBase`、`Health`、`StateMachine`，统一小怪、Boss、玩家的生命、受击、死亡和动画桥接。
- **关卡闭环**：建立`Room`、`Transition`、`Checkpoint`、`Gate`、`MapMarker`，支撑房间切换、能力锁、检查点和地图。
- **资源目录**：按`Combat`、`Enemies`、`Items`、`Rooms`、`Audio`、`UI` 分组，避免资源增长后难维护。

3 目标架构

3.1 架构原则

1. **组合优先于继承**：玩家、小怪、Boss 共用`Health`、`Hurtbox`、`Hitbox`、`StateMachine`、`AnimationBridge`，而不是建立巨大继承树。
2. **数据资源驱动**：攻击参数、敌人参数、道具定义、房间元数据尽量用Godot `Resource` 管理。
3. **节点负责表现，组件负责规则**：`Area2D`、`CharacterBody2D` 负责场景行为；伤害、血量、存档、能力解锁等规则尽量独立。
4. **核心层不依赖具体玩法层**：`Core` 提供事件、存档、场景切换、配置，但不直接引用`Player` 或`Enemy`。
5. **每两周都可运行**：不要等内容完成才处理导出、设置、输入映射、存档和QA。

3.2 推荐目录结构

```
HollowKnight/
  Scenes/
    Boot.tscn
    GameRoot.tscn
    Test/
      CombatSandbox.tscn
      MovementSandbox.tscn
    Player/
      Player.tscn
    Enemies/
      Slime.tscn
      FlyingEnemy.tscn
      MiniBoss_01.tscn
    Level/
      Rooms/
        Room_001_Start.tscn
        Room_002_Combat.tscn
      Props/
      Transitions/
    UI/
```

```
HUD.tscn
PauseMenu.tscn
MainMenu.tscn
```

Scripts/

Core/

```
GameRoot.cs
SignalBus.cs
SceneLoader.cs
InputBindings.cs
Save/SaveManager.cs
Save/SaveData.cs
Debug/DebugOverlay.cs
```

Combat/

```
DamageInfo.cs
DamageTeam.cs
IDamageable.cs
Hitbox.cs
Hurtbox.cs
HealthComponent.cs
AttackProfile.cs
InvincibilityComponent.cs
KnockbackComponent.cs
```

Entities/

```
EntityBase.cs
EnemyBase.cs
BossBase.cs
StateMachine/State.cs
StateMachine/StateMachine.cs
```

Player/

```
PlayerController.cs
PlayerMotor.cs
PlayerCombat.cs
PlayerAbilities.cs
PlayerAnimationBridge.cs
```

Level/

```
Room.cs
RoomTransition.cs
Checkpoint.cs
CameraBounds.cs
SpawnPoint.cs
AbilityGate.cs
MapMarker.cs
```

UI/

```
HudController.cs
MenuController.cs
SettingsPresenter.cs
```

Resources/

```
Combat/AttackProfiles/
```

```
Enemies/
```

```
Items/
```

Rooms/
Audio/
UI/

Tests/
Exports/
Docs/

4 模块路线

4.1 Core 模块

组件	职责	阶段
GameRoot	游戏入口、服务注册、运行状态、调试开关	P0
SignalBus	跨模块事件：死亡、房间切换、道具获取、存档完成	P0-P1
SceneLoader	加载主场景、测试场景、房间切换	P2-P3
SaveManager	保存玩家状态、房间状态、检查点、设置	P3
DebugOverlay	FPS、当前房间、玩家状态、碰撞盒开关	P1

4.2 Combat 模块

建议将当前Hurtbox.cs 从“直接扣血”改为“伤害请求+ 组件委托+ 可观测事件”。

代码示例：推荐的伤害数据结构

```
using Godot;

public readonly struct DamageInfo
{
    public readonly int Amount;
    public readonly Vector2 Knockback;
    public readonly Node2D Source;
    public readonly DamageTeam Team;

    public DamageInfo(int amount, Vector2 knockback, Node2D source, DamageTeam team)
    {
        Amount = amount;
        Knockback = knockback;
        Source = source;
        Team = team;
    }
}

public enum DamageTeam
{
    Player,
    Enemy,
    Neutral
}
```

```
public interface IDamageable
{
    void TakeDamage(DamageInfo info);
}
```

代码示例: Hurtbox 推荐骨架

```
using Godot;

public partial class Hurtbox : Area2D, IDamageable
{
    [Signal]
    public delegate void ReceivedDamageEventHandler(int amount, Vector2 knockback);

    [Export] public NodePath HealthComponentPath { get; set; }

    private HealthComponent _health;

    public override void _Ready()
    {
        if (HealthComponentPath != null && !HealthComponentPath.IsEmpty)
            _health = GetNode<HealthComponent>(HealthComponentPath);
    }

    public void TakeDamage(DamageInfo info)
    {
        _health?.ApplyDamage(info.Amount);
        EmitSignal(SignalName.ReceivedDamage, info.Amount, info.Knockback);
    }
}
```

注意: Godot C# Signal 参数适合使用int、float、string、Vector2、Node 等Variant 兼容类型。DamageInfo 可用于内部调用, 但不建议直接作为Godot Signal 参数。

脚本/资源	职责	验收标准
DamageInfo	单次伤害请求的数据载体	支持伤害值、击退、来源、阵营
AttackProfile	攻击配置资源: 伤害、硬直、CD、击退、特效	不改代码即可调整普通攻击参数
Hitbox	攻击判定区域, 检测Hurtbox 并发送伤害	能区分玩家/敌人阵营, 避免友伤
Hurtbox	被攻击判定区域, 接收伤害请求	支持无敌帧、受击动画、击退
HealthComponent	血量、死亡、回血、伤害应用	玩家/敌人共用, 死亡事件可订阅
InvincibilityComponent	无敌帧、闪烁、重复受击冷却	避免单次攻击多次误伤
KnockbackComponent	受击速度覆盖、恢复、抗击退倍率	地面/空中击退可分别调参

4.3 Player 模块

脚本	职责	关键参数
PlayerController	总协调: 输入、状态、Motor、Combat、Abilities	当前状态、是否可控制
PlayerMotor	跑、跳、重力、冲刺、平台边缘容错	加速度、最大速度、土狼时间、跳跃缓冲
PlayerCombat	普攻、下劈、蓄力、攻击CD、连击窗口	攻击帧、命中停顿、后摇、Hitbox 激活时间
PlayerAbilities	二段跳、冲刺、爬墙、特殊攻击、能力门	已解锁能力、冷却、消耗
AnimationBridge	把状态映射到动画树或AnimatedSprite	动画名、过渡、受击/死亡优先级

4.4 Entities 模块

敌人不要一开始追求复杂AI。优先实现稳定的3 层结构:

- 1. **EntityBase**: 生命、朝向、死亡、通用组件引用。
- 2. **EnemyBase**: 玩家检测、仇恨、巡逻、攻击入口。
- 3. **具体Enemy**: 只写差异行为, 例如地面巡逻、飞行追踪、远程攻击。

敌人类型	目的	实现重点	阶段
地面巡逻怪	验证移动、碰撞、受击、死亡	巡逻边界、掉头、短距离攻击	P2
飞行追踪怪	验证空中敌人和追踪算法	简单steering、避障、接近攻击	P2–P3
炮台/远程怪	验证投射物系统	Projectile、阵营、消失、命中反馈	P3
小Boss	验证阶段切换和复杂动画	3 个招式、血量阶段、攻击前摇提示	P4

4.5 Level 模块

关卡模块建议从房间制开始, 而不是一开始构建超大无缝地图。

脚本/资源	职责	验收标准
Room	房间ID、出生点、敌人刷新、房间状态	能在编辑器中配置房间元数据
RoomTransition	门/边界切换, 指定目标房间和出生点	切换后玩家位置、朝向、相机边界正确
Checkpoint	存档点、复活点、回血、存档触发	死亡后回到最近检查点
CameraBounds	当前房间相机限制	不露出关卡外黑边
AbilityGate	需要能力才能通过的门或机关	未解锁能力时有明确反馈
MapMarker	地图格子、门、宝箱、检查点信息	UI 地图能读取并显示

4.6 UI 模块

UI 的第一目标不是美观，而是服务测试和可用性。推荐顺序：HUD、暂停菜单、设置菜单、地图界面、标题界面。

5 模块依赖规则

硬规则：Core 不能引用Player、Enemy、UI 的具体类；Combat 不直接控制UI；Level 可以触发事件但不直接修改玩家内部状态；UI 只订阅状态或事件，不承载游戏规则。

模块	Core	Combat	Entities	Player	Level	UI	说明
Core	–	否	否	否	否	否	只提供基础设施
Combat	可	–	否	否	否	否	不依赖具体角色
Entities	可	可	–	可少量	可少量	否	敌人可读取玩家位置，但避免强耦合
Player	可	可	否	–	可少量	否	玩家可响应关卡机制
Level	可	可少量	可少量	可少量	–	否	通过事件驱动房间状态
UI	可	可读	可读	可读	可读	–	UI 只展示，不拥有规则

6 AI 辅助 workflow

场景	可用方式	人工校验点
代码脚手架	生成组件骨架、接口、Resource 类、测试场景脚本	Godot 生命周期、Signal、NodePath、Variant 类型兼容
策划表	生成敌人参数、攻击参数、关卡节点清单	数值必须通过试玩调参
像素占位图	生成概念图、道具图标、UI 占位、氛围图	统一分辨率、轮廓、调色板、版权风险
动画参考	生成动作分解、关键帧参考、Boss 招式草案	最终像素帧仍需人工修正
关卡辅助	生成房间草图、节奏建议、能力门配置草案	路径、难度曲线、回溯体验需人工试玩
文档与测试	生成Sprint 清单、测试用例、提交说明、Bug 模板	不能替代真实QA 和玩家反馈

AI 资产进入项目建议设置门禁：任何AI 图像先进入Resources/Concept 或Resources/Placeholder；最终像素资产必须统一尺寸、调色板、描边、光源方向、透明边缘和导入预设；Prompt、模型、生成时间、用途记录在Docs/AIAssetLog.md。

7 进度规划：24 到30 周垂直切片

7.1 产能假设

- 团队：3 名C# 程序员兼职。
- 单人每周有效产能：8–12 小时。
- 团队每周有效产能：24–36 小时。
- 2 周为一个Sprint：约48–72 团队小时。
- 规划按可持续兼职估算，未把返工、生活波动、学习成本理想化。

7.2 成员分工建议

成员	主责	典型任务	备份
程序A	Core / Build / Save / 工具	项目规则、CI、场景切换、存档、DebugOverlay、导出	Level
程序B	Player / Combat	移动手感、攻击、Hitbox/Hurtbox、受击、Boss 战斗逻辑	Entities
程序C	Level / UI / Content Pipeline	房间、相机、检查点、地图、HUD、资源导入流程	Core

7.3 阶段目标

阶段	目标	交付物	建议时长
P0 架构基线	统一项目规则、目录、场景命名、输入映射	可运行主场景、基础分层、分支策略	1–2 周
P1 动作原型	主角移动、跳跃、冲刺、攻击、受击、击退	测试房间中可战斗，帧感可调	3–6 周
P2 战斗闭环	打通Hitbox/Hurtbox/Health/Enemy	2 种小怪、基础HUD、死亡反馈	5–10 周
P3 关卡闭环	房间切换、相机边界、检查点、存档、地图	4–6 个连续房间，可死亡重生	8–16 周
P4 垂直切片	20–30 分钟试玩路线	小Boss、能力解锁、菜单、音效、打包	16–30 周
P5 对外Demo	Steam/玩家测试级Demo	新手引导、设置、键鼠/手柄、QA、性能稳定	30–52 周

7.4 Sprint 计划

周	工作与验收
1–2	P0 架构基线： 建立目标目录，调整mainScene 为Boot/GameRoot，配置输入映射和分支策略。 验收： 任意成员拉库后可运行，主场景可进入测试房间。

3-4	P1 移动原型: PlayerMotor 实现跑、跳、二段跳占位、土狼时间、跳跃缓冲、基本相机。 验收: 参数可在Inspector 调整, 5 分钟测试无明显卡墙。
5-6	P1 战斗原型: Hitbox、Hurtbox、Health, 普攻、下劈占位、受击击退、无敌帧、命中停顿。 验收: 玩家可击杀测试假人, 反馈包含闪烁、击退、音效占位。
7-8	P2 小怪闭环: 地面怪、飞行怪、巡逻、追踪、死亡、掉落占位。 验收: 2 种敌人可稳定战斗, 敌人不频繁卡地形。
9-10	P2 UI 与调试: HUD、DebugOverlay、暂停菜单、碰撞盒开关、当前状态显示。 验收: 可看到血量、FPS、房间、玩家状态。
11-12	P3 房间系统: Room、Transition、SpawnPoint、CameraBounds。 验收: 至少3 个房间可互相切换, 相机边界正确。
13-14	P3 检查点与死亡: Checkpoint、死亡流程、重生、房间状态重置策略。 验收: 死亡后回检查点, 血量、位置、敌人状态符合预期。
15-16	P3 存档雏形: SaveData、SaveManager、设置保存、已探索房间、能力解锁状态。 验收: 关闭游戏重开后能继续到最近检查点。
17-18	P3 地图与能力门: 地图UI、MapMarker、AbilityGate、基础回溯路线。 验收: 获取能力后能打开新路径, 地图能显示当前房间。
19-20	P4 小Boss 原型: MiniBoss, 3 个招式、血量阶段、攻击前摇、死亡奖励。 验收: Boss 可被击败, 招式可读、可躲、可调参。
21-22	P4 内容串联: 4-6 个房间、2 种小怪、1 个能力、1 个检查点、1 个Boss。 验收: 形成20-30 分钟可玩路线。
23-24	P4 音效/美术占位统一: 替换粗糙占位, 统一像素导入, 加入基础音效和BGM 占位。 验收: 玩家不需要解释即可理解攻击、受击、门、检查点。
25-26	P4 试玩与返工: 5-10 人封闭试玩, 记录Bug、手感反馈、迷路点。 验收: 关键Bug 修复, 战斗和路径可被普通玩家完成。
27-30	P5 Demo 打包: 主菜单、设置、键位提示、导出Windows 包、README、Bug 收集表。 验收: 外部玩家可独立运行, 无需开发者口头说明。

7.5 每个Sprint 的固定节奏

时间点	动作
周一/周二	明确本Sprint 唯一主目标, 不超过3 个大任务
周中	每人提交至少1 次可运行变更, 避免周末才合并
周末	集成测试, 录制3-5 分钟试玩视频, 更新Bug List
Sprint 结束	只看可玩结果, 不按代码行数评估进展

8 质量门槛

类别	验收条件	最低标准
移动手感	起跳、落地、转向、冲刺、平台边缘容错自然	5 名玩家中至少3 名认为基本顺手

战斗反馈	攻击命中有停顿、音效、受击动画、击退、伤害反馈	玩家能明确判断是否命中
敌人行为	小怪有可读前摇和稳定碰撞	10 分钟试玩不大量卡死
关卡路径	房间切换、相机边界、检查点、回溯路线可理解	玩家无需解释能完成主路线
存档	保存检查点、血量、能力、已探索房间	关闭重开后不丢关键进度
导出	Windows 包能在非开发机器运行	无需安装Godot编辑器
性能	Demo 场景稳定运行，无明显卡顿	目标60 FPS，调试时可显示FPS

继续扩大开发的闸门：第24–30 周必须做一次立项复盘。若垂直切片无法达到“外部玩家独立跑完20–30 分钟”的水平，不建议继续扩写完整地图，应先压缩范围、重构战斗或补足美术流程。

9 主要风险与控制措施

风险	等级	表现	控制措施
战斗手感不过关	高	参数很多但不好玩，攻击无反馈，受击不清晰	前6 周只做手感；每周录屏对比；引入命中停顿、屏震、音效
内容规模失控	高	房间、怪物、Boss、剧情不断增加	先锁定4–6 房间垂直切片；完整游戏另行立项
美术产能不足	高	AI 图不统一，像素动画返工巨大	占位图用到P4；最终资产先只做核心角色；建立调色板和Style Sheet
Godot 场景耦合	中	NodePath 到处硬连，移动节点就报错	组件化；导出引用；统一事件；测试场景独立验证
C# Signal/Variant 坑	中	自定义结构传信号失败，运行期才暴露	Signal 使用Variant 兼容参数；复杂数据走方法或C# event
兼职节奏断裂	高	几周无人集成，最后无法合并	2 周Sprint；每周可运行；只接受小PR
Demo 质量不足	中	玩家无法独立理解玩法	Demo 前做封闭测试；必须有教程提示、设置、键位

10 近期4 周行动清单

10.1 第1 周

- 把mainScene.tscn 改为明确入口：Boot.tscn 或GameRoot.tscn。
- 建立Scenes/Test/CombatSandbox.tscn 与Scenes/Test/MovementSandbox.tscn。
- 增加Scripts/Core/DebugOverlay.cs，显示FPS、当前房间、玩家状态。
- 完成DamageInfo、DamageTeam、HealthComponent 草稿。

10.2 第2 周

- 修正Hurtbox.cs: 显式声明ReceivedDamage Signal; 委托给HealthComponent。
- 新增Hitbox.cs, 让攻击区域能够找到IDamageable 并调用TakeDamage。
- 做一个测试假人: 被打会扣血、闪烁、打印日志、死亡消失。
- 提交可运行版本并录制1 分钟战斗判定测试视频。

10.3 第3 周

- 拆出PlayerMotor.cs: 跑、跳、重力、落地检测。
- 加入Inspector 可调参数: 最大速度、加速度、跳跃速度、重力倍率、土狼时间、跳跃缓冲。
- 统一输入映射: MoveLeft、MoveRight、Jump、Attack、Dash、Interact、Pause。

10.4 第4 周

- 接入PlayerCombat.cs: 普通攻击、攻击冷却、Hitbox 激活窗口。
- 加入命中停顿和轻微屏震占位。
- 建立第一版参数表: 玩家移动参数、攻击参数、敌人血量参数。

11 协作与分支策略

11.1 分支建议

- master/main: 只保留稳定可运行版本。
- develop: 日常集成分支, 每周至少稳定一次。
- feature/player-motor、feature/combat-hitbox: 小功能分支, 尽量1-3 天合并。

11.2 Godot 场景协作规则

1. 同一时间只允许一个人编辑同一个.tscn 主场景。
2. 场景尽量拆小: 玩家、敌人、房间、UI 分别独立场景, 再实例化组合。
3. 文本资源开启版本控制; 大型音频、美术资源建议使用Git LFS 或外部资产管理方案。
4. 每次PR 必须说明: 新增节点、修改脚本、影响场景、测试方法。

12 最终建议

以当前架构为起点, 最优路径是: **先完善Combat + Player + Level 的最小闭环, 再做内容扩展**。前30 周的唯一目标应是一个小而完整的垂直切片, 而不是完整类《空洞骑士》规模的地图。只要每2 周都能交付可运行版本, 当前目录结构可以支撑MVP; 若模块之间开始互相直接引用、场景越做越大、没有测试房间和调试工具, 项目很快会进入不可维护状态。
